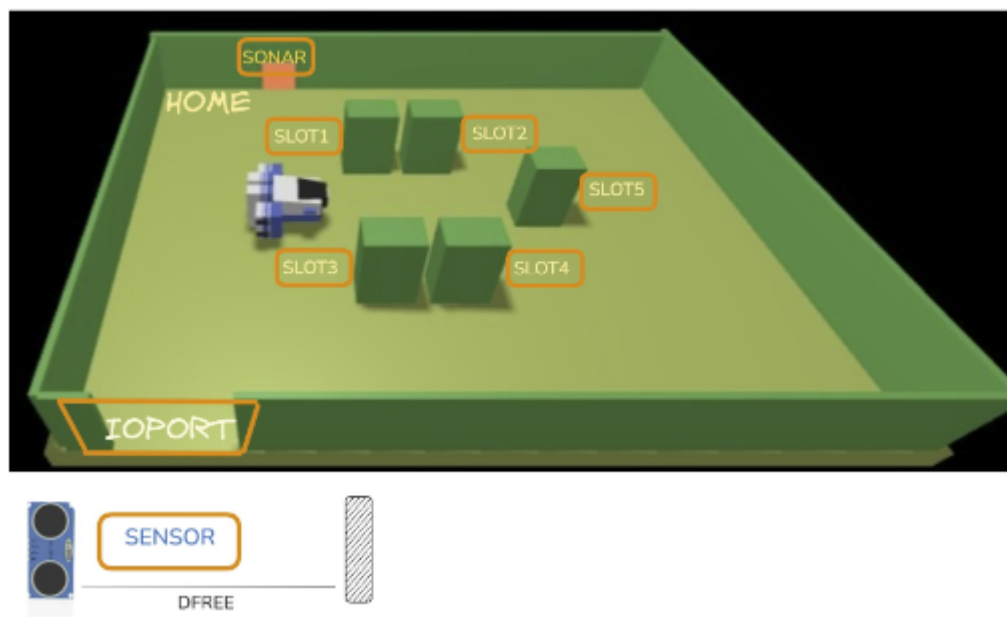


Introduction

Una compagnia di spedizioni marittime di merci (d'ora in poi, semplicemente compagnia) intende automatizzare le operazioni di carico dei container nella **stiva** della nave (o semplicemente stiva). A tal fine, la compagnia prevede di impiegare un Robot a Guida Differenziale (d'ora in poi, chiamato **cargorobot**). La stiva è un'area piana e rettangolare dotata di una porta di Input/Output (**IOPort**). L'area fornisce **4** slot per lo stoccaggio dei container e uno slot denominato *slot5*.



Nell'immagine sopra:

- Gli **slot1-4** rappresentano le aree della stiva riservate per stoccare un container ciascuna.
- Lo **slot5** rappresenta un'area in cui il *cargorobot* deve depositare temporaneamente un container, prima di collocarlo in uno degli slot1-4. Durante lo stoccaggio temporaneo, un dispositivo 'marker' (marcatore) etichetta il container con un codice a barre identificativo e segnala quando questa attività di marcatura è completata.

- L'**IOPort** è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display è utilizzato per mostrare la risposta alla richiesta e per visualizzare lo stato attuale della stiva.
- Il **sensore** associato all'*IOPort* è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D tale che $D < D_{FREE}/2$, per un tempo ragionevole (es. **3** secondi).

Requirements

La compagnia ci chiede di costruire un servizio chiamato **cargoservice** che dovrebbe funzionare come segue.

Il *cargoservice* è in grado di ricevere una richiesta di carico di un container inviata da un cliente utilizzando il pulsante dell'*IOPort*.

- Invia la risposta *retrylater* (riprova più tardi) se l'*IOPort* è attualmente occupata da un container o se il sistema è *Out of service* (fuori servizio).
- Rifiuta la richiesta quando la stiva è già piena, ovvero gli slot1-4 sono già occupati.
- Altrimenti, considera il sistema come **engaged** (impegnato), individua uno slot libero e restituisce come risposta il nome di tale slot riservato. Mentre è impegnato, il sistema deve far lampeggiare un Led.

Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (es. **30** secondi), altrimenti il sistema diventa *disengaged* (disimpegnato).

Successivamente, il *cargoservice* utilizza il *cargorobot* per spostare il container dall'*IOPort* allo *slot5* (per la marcatura del container) e poi allo slot riservato. Il servizio deve inoltre mostrare sul display dell'*IOPort*:

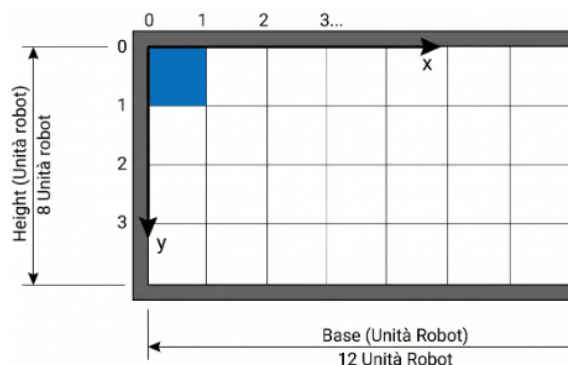
- lo stato attuale della stiva
- il messaggio '**Service working**', quando tutto procede regolarmente
- il messaggio '**Out of service**' se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno **3** secondi (segnalando forse un guasto del sonar).

Requirement analysis

1. Hold e Slot:

I requisiti richiedono la gestione dello spazio fisico della stiva (*Hold*) e dei relativi sotto-spazi (*Slots*).

- **L'Unità Robotica:** Definita la dimensione del DDR come un quadrato di lato L in cui il robot stesso è inscrivibile, tale dimensione viene assunta come l'unità di misura spaziale del sistema.
- **Modello della Hold:** In ottemperanza al testo, la stiva presenta larghezza e lunghezza esprimibili in unità robotiche. Pertanto, la *Hold* viene formalizzata logicamente come una **griglia discreta (matrice)** divisa in celle di dimensione L. In posizioni specifiche di questa griglia (indicate da coordinate logiche X, Y) sono posizionati gli Slot e la porta di Input/Output (*IOPort*).



```
public class Hold {  
    private int D           //dimensione unità robotica in metri  
    private int width;      // larghezza hold (in unità robotiche)  
    private int length;     // lunghezza hold (in unità robotiche)  
    private int[][] grid;   //griglia hold
```

}

- **Vincolo di Capacità e Slot 1-4:** Il sistema ha un limite quantitativo rigido. La capienza massima è di 4 container. Ognuno degli slot 1-4 rappresenta una zona quadrata di un'unità robotica adibita allo stoccaggio permanente. Ogni slot è un'entità logica caratterizzata da uno stato (Liberi o Occupati).
- **Slot 5:** Anch'esso una zona quadrata di un'unità robotica, ma non funge da stoccaggio permanente. Serve esclusivamente come area temporanea per la fase di *barcode labeling* prima che il container venga trasferito nel suo slot riservato.

```
public class Slot {  
    private String name;           // "SlotN"  
    private boolean occupato = false; // stato dello slot (occupato  
    o libero)  
}
```

- **Tracciamento dello Stato:** In base ai requisiti, cargoservice deve mantenere costantemente traccia dello stato di occupazione di ciascuno slot per poter implementare correttamente la logica di accettazione o rifiuto delle nuove richieste di carico.

2. DDR/CargoRobot:

Al fine di chiarire i vincoli tecnologici e operativi legati all'unità mobile, è stato effettuato un confronto diretto con il committente:

Domanda al committente: Il software di controllo di basso livello del DDR viene fornito come asset preesistente o deve essere interamente realizzato da noi?

Risposta: La software house ha già a disposizione un software pregresso che si presenta come un servizio per la gestione del *Differential Drive Robot*, denominato `robotsmart26`. Questo componente fornisce già un'interfaccia nativa basata su comandi in linguaggio ARIL (*Abstract Robot Interaction Language*).

Vincolo di Interfaccia Esterna (Dati di Input)

L'esistenza di `robotsmart26` rappresenta un vincolo di progetto esogeno. Tale componente impone che l'interazione con l'attuatore fisico avvenga rigorosamente tramite lo scambio esplicito di messaggi. Per il controllo del robot e del suo stato, l'interfaccia fornita dal committente è la seguente:

```
// --- Interfaccia con robotsmart26 (dato di input preesistente) ---
Event    alarm                : alarm(X)                "event at applicati
on level"
Dispatch move                  : move(M)                "MOVE = l|r|a|d|h
mosse ARIL sincrone"
Dispatch setrobotstate        : setpos(X,Y,D)           "set robot position
to (X,Y) direction D=up|down|left|right"
Request  setdirection         : dir(D)                 "set robot directio
n to D=up|down|left|right"
Reply    setdirectiondone     : pos(PX,PY)             for setdirection
Dispatch changedir           : changedir(X)           "autodispatch inter
no"
Request  getrobotstate        : getrobotstate(ARG)     "request robot stat
e ARG unused"
Reply    robotstate           : robotstate(POS,DIR)    for getrobotstate
"POS->pos(X,Y) DIR->up|down|left|right"
```

Nello specifico, il payload associato al comando `move` utilizza il linguaggio **ARIL**, il quale fornisce un vocabolario standard per esprimere i comandi base di spostamento discreto del robot nell'ambiente.

Si può notare che l'uso di RobotSmart26 introduce la necessità di definire esplicitamente i messaggi di comando, con cui è possibile interagire con il robot stesso, rendendo evidente come l'applicazione interagisce con il robot. Per questi motivi, robotsmart26 rientra nella categoria di sistemi a **microservizi**

4. CargoService

L'entità *CargoService* funge da orchestratore centrale del sistema. Il suo ruolo principale è custodire lo stato applicativo globale, ricevere ed elaborare le richieste di carico provenienti dall'esterno e inviare i comandi operativi al robot per la movimentazione dei container.

Dal punto di vista comportamentale, come delineato dai requisiti del committente, l'orchestratore possiede una duplice natura:

- **Proattiva:** esegue attivamente le sequenze di pianificazione e coordina le fasi di movimento del robot quando una richiesta viene accettata.
- **Reattiva:** rimane costantemente in ascolto di eventi esogeni, come lo scorrere

del tempo (timeout), messaggi di allarme o variazioni di stato dei sensori ambientali.

Dai requisiti inoltre si può notare che il sistema nasce già in un contesto distribuito. L'interazione via chiamata a procedura non è praticabile, pertanto CargoService viene modellato come microservizio: un componente software indipendente che comunica esclusivamente tramite scambio di messaggi.

Il modello computazionale naturale per realizzare microservizi che interagiscono tramite messaggi è il **modello ad attori**. Le entità che interagiscono nel sistema vengono formalizzate come attori, in quanto presentano le seguenti caratteristiche:

- Ogni attore è un'entità completamente incapsulata e in quanto tale lo stato e la struttura interna non sono visibili dall'esterno, pertanto l'unico modo per interagire con un attore è tramite scambio di messaggi asincroni. In questo modo la logica si sposta sul contratto di interazione tra gli attori. Si supera così la sincronizzazione tra le parti interagenti imposta dal meccanismo di trasferimento del controllo tipico della chiamata a procedura dell'OOP, fornendo al sistema disaccoppiamento e flessibilità grazie all'interazione asincrona tramite scambio di messaggi.
- Gli attori permettono di gestire la concorrenza in modo naturale, modellando ogni entità con comportamento autonomo come un componente indipendente che si coordina con gli altri esclusivamente tramite scambio di messaggi, senza condivisione di stato.

Modello dell'Interazione

Il *cargoservice* riceve le richieste di carico di un container inviate dal cliente tramite il pulsante dell'*IOPort*. Pertanto, il servizio viene interrogato dall'*IOPort* attraverso una specifica richiesta asincrona (*loadRequest*).

A questa richiesta, l'orchestratore associa una risposta diversa valutando rigorosamente il proprio stato interno:

- **loadEngaged**: Il sistema ha capienza sufficiente ed è disponibile per accettare una nuova richiesta. Comunica l'avvenuta presa in carico e lo slot riservato.
- **loadRejected**: La richiesta viene respinta in via definitiva poiché la stiva ha raggiunto la capienza massima consentita (4 container).
- **retryLater**: La richiesta viene momentaneamente rifiutata poiché l'*IOPort* è correntemente occupata o il sistema è impegnato in altre operazioni critiche.

L'utente è invitato a riprovare più tardi.

Gestione del Tempo e degli Eventi Sensoriali

Il *cargoservice* deve garantire il rispetto dei vincoli temporali previsti per il trasferimento del container nella sensor area (la regola dei 30 secondi). Se il tempo limite scade, l'orchestratore forza il ritorno del sistema allo stato *disengaged* e ne informa contestualmente *IOPort* attraverso un messaggio asincrono (*Event timeout*), affinché l'interfaccia utente possa aggiornarsi.

A completamento della sua natura reattiva, l'orchestratore gestisce in modo asincrono i dati provenienti dall'ambiente fisico. Nello specifico, viene informato l'avvenuto posizionamento di un container nella zona di carico tramite un evento emesso dal sensore (*containerDetected*). Analogamente, qualora il sonar rilevi un'anomalia prolungata (sintomo di un guasto). Il CargoService intercetta l'evento di allarme(*sensorAlarm*) e reagisce ponendo immediatamente il sistema in uno stato Out of service.

Interazione con il DDR

L'interazione tra *CargoService* e *RobotSmart26* avviene in modo asincrono tramite scambio di messaggi. Poiché il *CargoService* deve coordinare una sequenza operativa rigorosa (raggiungere l'IOPort, simulare la marcatura nello Slot 5 e depositare nello slot finale). Per fare ciò CargoService deve avere a disposizione l'interfaccia definita dal committente, con cui è possibile comunicare con il robot stesso.

Domanda al Committente: Dai requisiti si evince che c'è un led, dove si trova e come va realizzato?

Risposta: Il led può essere sia un led fisico, se disponibile oppure può essere un led virtuale che viene acceso o spento da qualche parte. Il posto in cui verrebbe acceso è la stessa pagina web precedente.

Il cargoservice deve poter iniziare o interrompere il LED (*toggleLedBlink*), sia che sia realizzato come fisico che come componente web. Nel caso sia web deve poter comunicare con la IOPort per riferire l'accensione del LED.

4. IOPort

Definizione e Ruolo Architettuale:

L'entità **IOPort** rappresenta il canale di comunicazione tra l'operatore umano (o sistemi esterni) e il CargoService. Ha la responsabilità bidirezionale di inoltrare i

comandi dell'utente verso il sistema logico (Input) e di mostrare i feedback visivi di ritorno (Output).

Domanda al committente: Il software che rappresenta la IOPort è già disponibile o dobbiamo costruirlo noi?

Risposta: Il componente deve essere sviluppato da zero. Sebbene sia possibile sviluppare l'**IOPort** come un pannello fisico reale (con pulsanti meccanici e schermi fisici), in questo progetto deve essere realizzata rigorosamente come un'interfaccia web (**Web GUI**). Si comporrà di due elementi virtuali: un pulsante (Push Button) che funge da dispositivo di input per l'invio della richiesta di carico, e un'area di testo dinamica (Display) che funge da dispositivo di output per mostrare lo stato del sistema all'utente.

Inoltre la pagina web con pulsante, display e LED virtuale, è un processo separato che gira nel browser di un nodo computazionale svincolato da quello di cargorobot.

5. Sensor

L'entità **Sensor** interagisce unicamente con CargoService con lo scopo di rilevare la presenza di un container da caricare, o, eventualmente, per comunicare un'anomalia critica che pone il sistema in stato di Out of Service. Come si evince dall'analisi del dominio, Sensor non ha rapporto o comunicazione con l'IOPort. È dotata di una duplice natura:

- **Reattiva:** Il sensore reagisce in tempo reale agli stimoli fisici dell'ambiente, rilevando le variazioni di distanza causate da un inserimento di un container o da un'ostruzione.
- **Proattiva:** Nel modello, il sensore non agisce come un componente passivo da interrogare (no polling), ma prende l'iniziativa. Valuta autonomamente i dati grezzi e, al soddisfacimento di una regola di business, genera e propaga tramite Broadcast un evento logico di alto livello.

I requisiti impongono regole di business precise basate su parametri quantitativi e temporali che il *Sensor* deve logicamente applicare. Data una distanza nominale a vuoto denominata DFREE:

1. **Rilevamento Container (`containerDetected`):** L'evento deve essere emesso esclusivamente se la distanza rilevata soddisfa la condizione $D < DFREE/2$ in modo stabile per un tempo $T \geq 3$ secondi.
2. **Allarme Guasto (`sensorAlarm`):** L'evento critico deve essere emesso qualora la distanza rilevata soddisfi la condizione $D > DFREE$ in modo continuo per un

tempo $T \geq 3$ secondi.

Per garantire il disaccoppiamento e gestire la sua natura asincrona, il Sensor deve essere formalizzato come un servizio a sé stante dedicato al filtraggio dei dati del dispositivo di rilevamento.

Struttura del sistema

La responsabilità di eseguire codice applicativo, non è attribuibile al nodo che realizza il CargoRobot, nè al nodo del sensore. Il sistema è quindi strutturato sui seguenti nodi computazionali:

1. Il nodo che supporta l'esecuzione del servizio fornito dal committente robotsmart26
2. Il nodo che supporta l'esecuzione del componente che realizza l'applicazione richiesta ossia cargoservice
3. Il nodo su cui, sfruttando il principio del disaccoppiamento, esegue il Sensor (come potrebbe essere un dispositivo PicoW).
4. Il nodo computazionale su cui esegue IOPort ossia la pagina web con pulsante, display e LED virtuale. CargoService interagisce con questo nodo attraverso il QActor IOPort.

Pertanto già dai requisiti si evince che il sistema è distribuito su quattro nodi, in cui i componenti sono logicamente (micro)servizi, possiamo formalizzare la struttura del sistema usando il linguaggio qak, fornito direttamente dalla software house. Questo linguaggio modellativo è stato sviluppato internamente per definire ed eseguire sistemi basati sullo scambio di messaggi, superando i limiti delle invocazioni dirette a metodi su nodi locali.



Modello QAK Analisi di Requisiti:

System cargosystem

Event	alarm	: alarm(X)	"Allarme emesso dal robot"
Dispatch	move	: move(M)	"MOVE = l r a d h mosse ARIL sincrone"
Dispatch	setrobotstate	: setpos(X,Y,D)	"Forza posizione (X,Y) e direzione D"
Request	setdirection	: dir(D)	"Richiesta rotazione D=up down left right"
Reply	setdirectiondone	: pos(PX,PY)	for setdirection
Request	getrobotstate	: getrobotstate(ARG)	"Richiesta stato corrente"
Reply	robotstate	: robotstate(POS,DIR)	for getrobotstate

```

// --- IOPort <-> CargoService ---
Request loadRequest : loadRequest(X)          "Richiesta di carico d
all'operatore"
Reply loadEngaged : loadEngaged(SLOT)         for loadRequest
Reply retryLater : retryLater(X)             for loadRequest
Reply loadRejected : loadRejected(X)         for loadRequest

// --- CargoService -> IOPort (Feedback Asincroni) ---
Event timeout : timeout(X)                    "Scadenza regola dei 3
0 secondi"
Event loadEnded : loadEnded(X)                "Fine procedura di car
ico"
Event endOOS : endOOS(X)                     "Fine stato Out Of Ser
vice"
Dispatch toggleLedBlink: toggleLedBlink(X)    "Comando LED on|off"

// --- Sensor -> CargoService (Eventi Proattivi di Business) ---
Event containerDetected : containerDetected(X) "Emesso se D < DFRE
E/2 per 3s"
Event sensorAlarm : sensorAlarm(X)           "Emesso se D > DFREE
per 3s"

// --- Contesti ---
Context ctxcargosystem ip [host="localhost" port=8020]
Context ctxrobotservice26 ip [host="localhost" port=8025]
Context ctxsensor ip [host="localhost" port=8135]

// Dichiarazione dell'entità esterna fornita dal committente
ExternalQActor robotsmart26 context ctxrobotservice26

QActor cargoservice context ctxcargosystem {
    State s0 initial {
        println("$name | starts - Inizializzazione Orchestratore") colo
r green
    }
    Goto idle
}

```

```

    State idle {
        // Stato di attesa reattiva delle richieste (loadRequest) o eventi (sensorAlarm)
    }
}

QActor ioport context ctxcargosystem {
    State s0 initial {
        println("$name | starts - Boundary Interfaccia Utente") color blue
    }
}

QActor sensor context ctxsensor {
    [# var DFREE #]

    State s0 initial {
        println("$name | starts - Boundary Sensore") color cyan
        // Il sensore applicherà logicamente le soglie DFREE per emettere eventi
    }
}

```

Problem analysis

Nel sistema proposto i (micro)servizi non verranno usati solo come modello formale e computazionale, ma anche come elemento organizzativo del software prodotto,

permettendo scalabilità, manutenibilità e flessibilità tecnologica.

Componenti forniti dal committente

Ci si chiede se il servizio robotsmart26 fornito dal committente sia sufficiente al fine di portare a termine i compiti richiesti dal sistema. Questo servizio ha la responsabilità di spostare il DDR in una posizione specificata. Attraverso una sequenza di messaggi move(M) e il linguaggio aril, cargoservice può comandare la sequenza di mosse con cui è possibile spostare il robot dalla IOPort allo Slot5 ed infine allo slot assegnato. Tuttavia questa pianificazione di mosse che servono per arrivare alla destinazione desiderata non deve essere responsabilità di cargoservice, il quale deve solo comandare affinché il ddr si sposti nella regione target.

In fase di analisi è stata introdotta la mappa della hold come griglia di interi. Il concetto di posizione viene formalizzato con una coppia di coordinate cartesiane che identifica una cella della matrice. Da questa visione è possibile generare una mappa logica della Hold che vede le posizioni occupate da ostacoli con il valore 1 mentre le posizioni libere con il valore 0. Un esempio potrebbe essere:

```
|r, 0, 0, 0, 0, 0
|0, 1, 0, 1, 0, 0
|0, 0, 0, 0, 1, 0
|0, 1, 0, 1, 0, 0
|0, 0, 0, 0, 0, 0
|0, 0, 0, 0, 0, 0
```

Dove r è la cella occupata dal robot, mentre gli 1 rappresentano le posizioni dello slot. Grazie a questo servizio pertanto il robot non si muove alla cieca; deve muoversi all'interno di una mappa topografica definita, riconoscendo le coordinate, le pareti e gli ostacoli dell'ambiente circostante

RobotSmart26 fornisce la possibilità di ottenere una mappa come matrice `int[ ][ ]` partendo da una stringa memorizzata nel file dato del tipo:

```
0000001@0011001@0000101@0011001@0000001@1111111
```

Robotsmart26 pertanto mantiene una mappa interna. Ci si chiede in questa fase se Cargoservice deve fornirla o se è già precaricata. In questa prima fase potrebbe essere semplicemente precaricata e fissa, impostando ostacoli e parti libere in modo fisso e a priori. Tuttavia si prevede in uno sviluppo successivo che sia il robot stesso a esplorare la stanza creandosi la mappa. Ci può essere un'applicazione chiamata

Room Explorer che comincia a esplorare in maniera sistematica la stanza all'avvio del sistema: dove è libera scriverà 0, dove è occupata scriverà 1. Il risultato prodotto è la stringa riportata sopra, considerando gli ostacoli come fissi.

Una volta formalizzata la mappa mentale, ci si chiede chi esegue la pianificazione delle mosse da fare per raggiungere lo slot assegnato. Anche in questo caso robotsmart26 espone un (nano)servizio detto planexec, in grado di eseguire una pianificazione automatica: lo scopo di planexec è eseguire una sequenza di mosse (un piano), data una destinazione finale e attraverso un'insieme di mosse base. L'interfaccia di messaggi fornita dal committente per poter interagire con questo servizio è la seguente:

```
Request doplan : doplan( PLAN, STEPTIME ) "execute PLAN with STEPTIME"
Reply doplandone : doplandone( ARG ) for doplan
Reply doplanfailed : doplanfailed( PLANTODO ) for doplan
Dispatch nextmove : nextmove(M) "autodispatch"
Dispatch nomoremove : nomoremove(M) "autodispatch"
Event alarm : alarm(X) "event at application level"
```

Questo servizio deve essere usato da cargoservice per la gestione del movimento del robot nella Hold, come verrà analizzato nello specifico con gli sprint successivi.

Hold e Slot

I requisiti definiscono la stiva (hold) come una griglia discreta di coordinate e gli Slot come sotto-spazi con uno stato binario (libero o occupato).

Come specificato precedentemente RobotSmart si aspetta una stringa composta nel seguente modo "0/1/X/@", pertanto al sistema verrà passata una stringa siffatta e questa verrà pertanto tradotta in una matrice del tipo: int[][] grid

Si decidono di formalizzare ulteriormente le caratteristiche desiderate da Hold attraverso un approccio Object-Oriented, modellando le due entità come classi (POJO) che incapsulano la logica spaziale. Ogni Coord rappresenta una cella quadrata di una unità robotica, cui dimensione in metri corrisponde ad un'area in cui è inscrivibile il robot stesso. Ad ogni Slot in quanto esistene nella Hold, è associata Coord all'interno della griglia. Allo stesso modo anche IOPort e Slot5 hanno le relative.

```
public class Slot {
```

```

    private String name;
    private boolean occupato = false;
    private Integer pid = null;    //pid del container assegnato (bar
code)
}

```

```

public class Hold {
    private int D;    //dimensione unità robotica in metri
    private int width;    // larghezza hold (in unità robotiche)
    private int length;    // lunghezza hold (in unità robotiche)
    private int[][] grid;    // griglia logica per la mappa mental
e

```

```

    private Map<Coord, Slot> gridSlots = new HashMap<>();

```

```

    private Coord ioport;
    private Coord slot5;

```

```

    public static class Coord {
        private final int x;
        private final int y;

        public Coord(int x, int y) {
            this.x = x;
            this.y = y;
        }

        public int getX() { return x; }
        public int getY() { return y; }
    }
}

```

IOPort

Per risolvere il problema del disaccoppiamento tecnologico del frontend, si decide in questa fase di modellare l'entità IOPort come un (micro)servizio, con cui è possibile comunicare mediante scambio di messaggi. Infatti, per evitare di rendere la logica applicativa del core business troppo dipendente dalla realizzazione tecnologica

dell'IOPort come WebGUI, viene formalizzata come un **QActor**.

Questa scelta ingegneristica di sviluppo incrementale permette di:

1. Astrarre il frontend.
2. Validare l'orchestratore in isolamento.

La WebGUI è erogata tramite server separato, attuando quindi una distinzione tra IOPort, ossia la macchina a stati, e guiserver, che eroga la pagina HTML che gestisce le connessioni con i vari utenti del sito.

Per la comunicazione tra WebGUI e IOPort si è scelto di utilizzare il protocollo WebSocket, in quanto permette di ricevere dati non appena sono pronti senza sovraccaricare il sistema. Questa scelta è necessaria in quanto cargoservice può emettere eventi in modo proattivo (e.g. `timeout`, `loadEnded` o `sensorAlarm`) che devono aggiornare lo stato del sistema (e quindi la GUI stessa) in tempo reale.

Una volta che cargoservice ha inviato messaggi al modello QAK di IOPort, i messaggi scambiati tramite WebSocket tra IOPort e WebGUI sono definiti con la seguente sintassi JSON:

```
{ "type": TYPE, "payload": PAYLOAD }
```

```
TYPE      ::= updateState | toggleLedBlink
```

```
PAYLOAD   ::= stateInfo | ledInfo
```

```
stateInfo ::= { "state": STATE, "message": MSG }
```

```
STATE     ::= disengaged | engaged | outofservice
```

```
MSG       ::= string
```

```
ledInfo   ::= { "active": ACTIVE }
```

```
ACTIVE    ::= true | false
```

Viceversa, i messaggi tra WebGUI e IOPort (ossia il solo comando del bottone premuto), sono definiti come segue:

```
{ "type": "buttonPressed" }
```

Questo messaggio arriva al guiserver via WebSocket. Il guiserver deve trasformarlo in un messaggio QAK e inviarlo al QActor ioport. Questo può portare a definire un `dispatch buttonPressed`, che una volta rilevato da IOPort, viene girato come una

Request a CargoService che provvederà a verificare la disponibilità di Slot, eventualmente aggiornando poi in base alla risposta lo stato presentato sul Display della Web GUI.

La precedente trattazione suggerisce un ulteriore possibile un test funzionale, al fine di validare il comportamento di cargoservice e IOPort **in isolamento**, senza browser né GUI, simulando il pushbutton tramite una richiesta HTTP diretta al guiserver.

Precondizione:

- cargoservice in stato idle
- almeno uno slot libero nella Hold
- guiserver attivo su `localhost:8040`

I comandi sono inviati, al momento, usando il tool curl:

```
curl -X POST http://localhost:8040/api/button \
  -H "Content-Type: application/json" \
  -d '{"type": "buttonPressed"}
```

Sequenza attesa osservabile:

1. guiserver riceve la richiesta HTTP e la traduce in `dispatch buttonPressed` verso il QActor ioport
2. ioport invia `loadRequest(X)` a cargoservice
3. cargoservice risponde con `loadEngaged(SLOT)`
4. ioport riceve `loadEngaged(SLOT)` e invia via WebSocket al browser (verificabile con un client WebSocket di test, es. `wscat`):

Sensor

In accordo con quanto detto fino ad ora, il sensore fisico che rileva la distanza degli oggetti rilevati, risulta un nodo a se stante, come potrebbe essere un dispositivo PicoW con un Sonar che misura la distanza fisica dagli oggetti. Il QActor `sensor`, in esecuzione su `ctxsensor`, costituisce il **boundary** tra il nodo fisico e il sistema applicativo: riceve le misurazioni grezze, applica le regole di business definite nei requisiti e propaga eventi logici verso cargoservice. Il protocollo di interazione con il sensore fisico non viene specificato in questa fase, per non essere vincolati a scelte tecnologiche vincolanti, tuttavia sembra opportuno citare come alternative valide TCP e MQTT.

Formalizziamo anche in questo caso tramite stringhe JSON i messaggi che emette un ipotetico sonar nella misurazione delle distanze:

```
{ "sensorName": "<sensorName>", "distance": <int> }
```

Il QActor Sensor, che conosce la distanza DFREE, una volta ricevuto un messaggio con il payload descritto, dovrà verificare tutte le condizioni specificate nei requisiti, producendo i seguenti event, definiti anche da CargoService:

```
Event    containerDetected : containerDetected(X) "Emesso se D < DFRE  
E/2 per 3s"  
Event    sensorAlarm      : sensorAlarm(X)      "Emesso se D > DFREE  
per 3s"
```

Si nota che il QActor sensor non è un semplice forwarder di messaggi, ma mantiene l'ultimo valore misurato e un timer di stabilità. Un singolo campione fuori soglia non è sufficiente a scatenare l'evento, ma la condizione deve persistere per la durata richiesta.

CargoService a sua volta userà queste informazioni per aggiornare lo stato del sistema, per gestire il movimento del ddr o per aggiornare la Web GUI.

Piano di lavoro e definizione degli sprint

Goal dello Sprint 1:

Si vuole realizzare il servizio cargoservice, in modo che gestisca correttamente lo stato della Hold e il dialogo con l'utente tramite IOPort (WebGui). In questa fase non viene ancora mosso fisicamente il robot ancora

Comprende:

- cargoservice riceve loadRequest da IOPort
- cargoservice valuta lo stato della Hold e risponde con `loadEngaged(SLOT)` , `loadRejected` , o `retryLater`
- IOPort (WebGui) mostra sul display la risposta ricevuta
- il LED inizia a lampeggiare quando il sistema è engaged
- cargoservice gestisce lo stato engaged/disengaged/pieno correttamente

Viene omessa la gestione del sensore, timer di 30s, movimento reale del robot.

Un primo piano ci vede convergere in 2 ulteriori sprint ossia:

Sprint2: integrare cargoservice con robotsmart26 per eseguire la sequenza di movimento reale IOPort → Slot5 → SlotRiservato, simulando l'arrivo del container con un trigger manuale (senza sensore fisico). Gestione del percorso da fare e del movimento del robot.

Sprint3: integrare il sensore (sonar su PicoW) nel sistema, abilitando il rilevamento reale del container e la gestione del timer dei 30 secondi. Verranno considerati in questa fase i guasti del sonar e le situazioni anomale.

Project

Testing

User Stories

- Come **Cliente**, voglio poter premere il pulsante dell'IOPort se il sistema è libero per richiedere il carico di un container, e sapere in quale slot riservato va depositato.
- Come **Cliente**, voglio sapere se la stiva di carico è piena al fine di non dover premere il pulsante ed aspettare una risposta
- Come **Cliente**, voglio sapere se il sistema è temporaneamente impegnato in un'altra operazione di carico, in modo da sapere se riprovare più tardi.
- Come **Cliente**, voglio vedere in ogni istante sul display lo stato attuale del sistema

- Come **Cliente**, voglio vedere un led lampeggiare quando il sistema è *engaged*.
- Come **Cliente**, mi aspetto un rifiuto immediato sul display se premo il pulsante mentre il sistema è engaged.
- Come **Cliente**, se non carico entro 30s il container nell'area del sensore, voglio poter essere avvisato che la richiesta è stata annullata ed il sistema è tornato disengaged.
- Come **Cliente**, voglio essere avvisato al termine della procedura di caricamento del container.
- Come **Cliente**, voglio sapere se il sistema è fuori servizio, per evitare di fare richieste inutili.
- Come **Sistema**, voglio rilevare automaticamente la presenza di container nell'area del sensore tramite quest'ultimo, in modo da poter avviare la sequenza per lo spostamento del container
- Come **Sistema**, voglio far stazionare il container nello slot 5 per il tempo necessario al barcode senza dover intraprendere ulteriori azioni.
- Come **Sistema**, voglio sapere quando il sonar smette di funzionare correttamente in modo da poter entrare nello stato di fuori servizio
- Come **Sistema**, analogamente voglio sapere quando il sonar riprende a funzionare, dopo un Out Of Service.
- Come **Sistema**, voglio monitorare il tempo che il cliente impiega a spostare il container nell'area del sensore.
- Come **Sistema**, voglio monitorare il tempo che il container trascorre nell'area del sonar

Test

In base all'interazione con l'utente richiesta dai requisiti, si definiscono i seguenti test funzionali:

TF1 - Richiesta accettata

Precondizione: sistema in stato *disengaged* , con almeno uno slot libero.

Input: IOPort invia `loadRequest(X)` a cargoservice

Risultato atteso:

- `loadEngaged(SLOT)` , con slot `SLOT` riservato per il container
- sistema in stato `engaged`
- led lampeggiante
- display mostra `SLOT` assegnato

TF2 - Richiesta rifiutata - hold piena

Precondizione: sistema `disengaged` , slot tutti occupati

Input: IOPort invia `loadRequest(X)` a cargoservice

Risultato atteso:

- sistema in stato `disengaged`
- `loadRejected(X)`

TF3 - Richiesta rifiutata - sistema già engaged

Precondizione: sistema `engaged`

Input: IOPort invia `loadRequest(X)` a cargoservice

Risultato atteso:

- `retryLater(X)`
- sistema rimane in stato `engaged`

TF4 - Timeout dopo 30 secondi

Precondizione: sistema entrato nello stato `engaged` , timer avviato

Input: dopo 30 secondi, se non viene ricevuto `containerDetected`

Risultato atteso:

- `timeOut` emesso verso IOPort
- led spento
- sistema entra nello stato `disengaged`

TF5 - Container rilevato e depositato

Precondizione: sistema `engaged`

Input: `containerDetected(X)`

Risultato atteso:

- il DDR si muove verso slot5, poi verso lo slot assegnato
- `loadEnded`
- led spento
- sistema `disengaged`

TF6 - Allarme sonar

Precondizione: stato `engaged` oppure `disengaged`

Input: `sensorAlarm(X)`

Risultato atteso:

- sistema entra nello stato di `outofservice`
- aggiornamento display
- led spento

TF7 - Fine Out of Service

Precondizione: sistema nello stato di `outofservice`

Input: `endOOS(X)`

Risultato atteso:

- sistema entra nello stato `disengaged`
- aggiornamento display

TF8 - Pressione pulsante durante engaged

Precondizione: sistema `engaged`

Input: pressione pulsante della WebGUI

Risultato atteso:

- nessun messaggio inviato
- aggiornamento display

Deployment

Maintenance